FULL-STACK APPLICATION / K12 HUNAR ASSESSMENT

EdTech CRM & Inquiry System

Detailed project report

Product scope, architecture, data model, security, and verified delivery.

PREPARED BY	ASSESSMENT	ORGANIZATION
Athwin Videsh M	Full-Stack Developer Intern First-Round Assessment	**K12 Hunar**

LIVE DEPLOYMENT	Frontend Netlify | API Render | Database Supabase PostgreSQL
REVIEWED	29 October 2026 | Production endpoints and current repository checked

Project access

PUBLIC EXPERIENCE	Open inquiry form	STAFF SIGN-IN	Open admin login
SOURCE REPOSITORY	Browse on GitHub	API HEALTH	Check production status

Project in one sentence

A public inquiry form feeds a PostgreSQL-backed Express API; authorized staff use a JWT-protected dashboard to search, filter, and update inquiry status. Administrative authentication is implemented in the backend with bcrypt and JWT, not Supabase Auth.

CONFIDENTIALITY
This report contains public project information only. It excludes database credentials, admin passwords, JWT secrets, and the original assessment brief PDF.

01 / PRODUCT

Purpose & scope

Centralize inquiry intake and give the team a clear, searchable place to follow up.

CHALLENGE	APPROACH	OUTCOME
Scattered inquiries Details can be hard to find and follow up on when intake is inconsistent.	**One shared workflow** A validated form, central records, and a private team dashboard.	**Visible progress** Staff can search, filter, and update the status of each inquiry.

Core capabilities

PUBLIC INQUIRY	ADMIN WORKSPACE
Collects user type, inquiry category, name, email, phone, and message. Phone numbers are validated as international numbers in the API.	Shows received inquiries with category and user-type filters, name/phone search, received date, and editable status.
AUTHENTICATION	**PERSISTENCE**
bcrypt password hashes and signed JWTs protect admin operations. Login requests are rate limited.	The API stores inquiry and admin records in Supabase-hosted PostgreSQL.

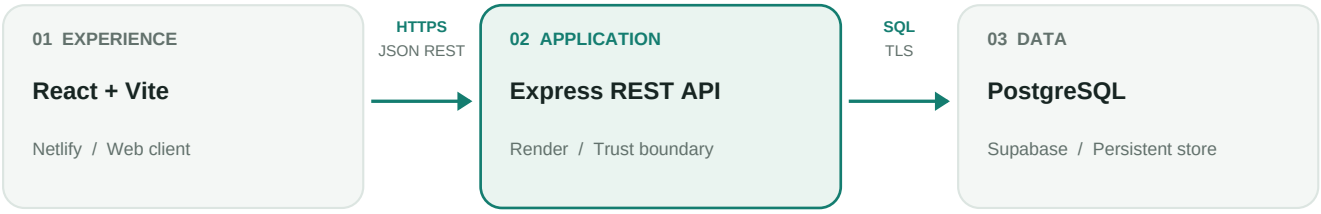
Primary user journey

01	**Submit** A visitor completes the inquiry form.
02	**Validate & save** The API validates the payload and writes the record.
03	**Authenticate** An admin signs in; the backend returns a signed token.
04	**Manage** The dashboard loads, filters, searches, and updates records.

Scope boundary
This is an inquiry intake and follow-up tool, not a learning-management system. Payments, course enrollment, automated outreach, analytics, and multi-role staff management are not represented as current features.

Architecture & API

Three clear layers. One server-side trust boundary. No direct database access from the browser.



API TRUST BOUNDARY Validates input, authorizes staff, and is the only layer that connects to PostgreSQL.

Responsibilities

LAYER	RESPONSIBILITY	DEPLOYMENT
Frontend	React UI, client-side form feedback, routing, dashboard interactions, and API requests.	Netlify
API	Express endpoints, request validation, login, JWT verification, rate limiting, and parameterized database operations.	Render
Database	PostgreSQL persistence for admin accounts and inquiries; schema constraints and lookup indexes.	Supabase

REST API

METHOD & PATH	ACCESS	PURPOSE
POST /api/inquiries	Public	Validate and save an inquiry; returns the new ID and creation time.
POST /api/auth/login	Public Rate-limited	Verify email/password and return an admin identity with a signed JWT.
GET /api/inquiries	Admin JWT	List newest first; optional category, userType, and name/phone search.
PATCH /api/inquiries/:id/status	Admin JWT	Change status to New, In Progress, or Resolved.
GET /api/health	Public	Check database connectivity and report API health.

Filter values and statuses are validated by the API; all inquiry-management routes require an admin JWT.

PostgreSQL schema

The SQL schema is the source of truth for field names, defaults, and allowed values.

admins

COLUMN	TYPE	RULE / DEFAULT
id	UUID	Primary key; generated with gen_random_uuid().
email	TEXT	Required and unique.
password_hash	TEXT	Required; stores bcrypt hash, not plaintext password.
created_at	TIMESTAMPTZ	Required; defaults to NOW().

inquiries

COLUMN	TYPE	RULE / DEFAULT
id	UUID	Primary key; generated UUID.
user_type	TEXT	Required; Parent, Student, or School.
category	TEXT	Required; Scholarship, Concert, Admission, or General Query.
name, email, phone, message	TEXT	All required. API validation adds format and length checks.
status	TEXT	Required; defaults to New; New, In Progress, or Resolved.
created_at	TIMESTAMPTZ	Required; defaults to NOW().
updated_at	TIMESTAMPTZ	Required; defaults to NOW(); updated on status change.

Integrity & retrieval

SCHEMA ELEMENT	PURPOSE
CHECK constraints	Restrict user_type, category, and status values to supported options.
Indexes	created_at DESC, category, and user_type for common dashboard sorting/filtering.
pgcrypto extension	Provides gen_random_uuid() for generated primary keys.

Open the source SQL: schema.sql.

04 / SECURITY

Authentication & safeguards

Authentication stays in the Express API; the browser never receives database credentials.

Admin session lifecycle

01 **Provision**
create-admin reads ADMIN_EMAIL and ADMIN_PASSWORD, hashes the password, and upserts the admin record.

02 **Sign in**
The API compares the submitted password with the stored bcrypt hash.

03 **Issue token**
On success, the API signs a JWT with the configured secret and expiry.

04 **Authorize**
The frontend stores the token in sessionStorage and sends it for private API calls.

05 **Verify**
Middleware verifies the token and confirms the admin record still exists. Sign-out clears sessionStorage.

Implemented protections

CONTROL	IMPLEMENTATION
Request validation	Zod schemas validate inquiry, login, status, and filter values; phone numbers use libphonenumber-js.
Authentication	bcryptjs password hashing; signed JWT; admin verification; missing/invalid token returns 401.
Abuse / browser	Login rate limit: 10 attempts per 15 minutes; Helmet security headers; CORS allows configured frontend origin.
Database access	Parameterized SQL values for inserts, login lookup, filters, and status updates.
Transport / payload	JSON body limit is 20 KB; production deployment uses HTTPS.

Security consideration
sessionStorage is cleared when the browser session ends, but any token accessible to page JavaScript can be exposed by cross-site scripting. Keep dependencies current and prevent unsafe script injection.

05 / OPERATIONS

Configuration & deployment

The frontend, API, and database are separately hosted and configured to communicate over HTTPS.

Environment configuration

VARIABLE	PURPOSE / HANDLING
DATABASE_URL	Supabase PostgreSQL URI; private. Retain the required SSL mode.
JWT_SECRET	JWT signing key; random, at least 32 characters, private.
JWT_EXPIRES_IN	JWT lifetime; optional, defaults to 8h.
PORT / NODE_ENV	API port (default 4000) and runtime mode.
FRONTEND_ORIGIN	Allowed web origin for CORS; defaults to localhost:5173.
ADMIN_EMAIL / ADMIN_PASSWORD	Inputs to create-admin; use a private admin password of at least 12 characters.
VITE_API_URL	Optional frontend API origin for split deployments; never place secrets in frontend variables.

Local start-up

1. Copy .env.example to .env; fill in private values and create a Supabase PostgreSQL database.
2. Run schema.sql in the Supabase SQL Editor.
3. From the repository root, run `npm install`.
4. Create the administrator: `npm run create-admin --workspace backend`.
5. Start both servers: `npm run dev`. Open localhost:5173; API health is localhost:4000/api/health.

Windows note
If PowerShell blocks npm.ps1, use npm.cmd. The create-admin command must run with the intended environment variables and database access.

Configuration reference: .env.example | README setup guide

06 / DELIVERY

Verification & next steps

Measured checks are separated from user flows that still need manual confirmation.

Verified on 29 October 2026

CHECK	RESULT	EVIDENCE
Backend test suite	PASS / 4 tests	Four validation-focused Node tests passed; not an end-to-end integration suite.
Frontend production build	PASS	Vite production build completed successfully.
Production API health	PASS	GET /api/health returned {"status":"ok"}.
Production frontend	HTTP 200	Public Netlify URL responded successfully.

Manual release checks

PUBLIC FORM	ADMIN FLOW	DATA MANAGEMENT
Submit a valid inquiry and confirm it is saved; verify invalid email/phone feedback.	Sign in with configured admin; confirm sign-out and unauthenticated dashboard redirect.	Verify filters/search, supported status changes, and unauthorized API access.

Future improvements

SCALE	OPERATIONS	PRODUCT
Pagination and server-side sorting for larger inquiry volumes.	Audit trail, structured logs, monitoring, and alerting.	Email follow-up reminders, analytics, and CSV export.

Summary
The project demonstrates a complete inquiry workflow using React, Express, PostgreSQL, bcrypt, and JWT. The API is the security boundary, the database schema enforces supported values, and production status and build checks were confirmed for this report.

Project references

REPOSITORY	GitHub source
README	Setup and project documentation
DATABASE	schema.sql
ENVIRONMENT	.env.example
SUPPORTING DOCS	Guides and inquiry examples
LIVE APP	Public inquiry form
LIVE API	Health endpoint

Project summary PDF: documents/report.pdf. The original assessment brief is intentionally excluded from the repository.